

Отчет

appointments.py

Этот код:

1. Принимает POST-запросы на создание записи.
2. Валидирует входные данные через Pydantic.
3. Сохраняет данные в БД через SQLAlchemy.
4. Возвращает созданный объект в стандартизированном формате (AppointmentOut).

Плюсы:

1. Чистая архитектура - Код разделён на роутеры, модели и схемы, что соответствует принципам FastAPI.
2. Использование Dependency Injection - get_db используется через Depends, что правильно для управления сессиями БД.
3. Автоматическая валидация - Используются Pydantic схемы (schemas.AppointmentCreate, schemas.AppointmentOut).
4. Корректное управление сессией БД - Сессия открывается и закрывается в finally блоке.
5. Соответствие REST - POST-эндпоинт возвращает созданную сущность с кодом 200 (по умолчанию в FastAPI).

Минусы:

1. Нет обработки ошибок (например, если БД недоступна).
2. Нет проверки на дубликаты или бизнес-логики (например, можно ли записаться к этому врачу в это время?).
3. Используется синхронный подход (лучше перейти на асинхронный, если БД поддерживает).

Как можно улучшить:

1. Добавить обработку ошибок (try-except).
2. Валидировать бизнес-правила (например, доступность времени у врача).
3. Использовать асинхронный подход (async/await).
4. Добавить логирование.

doctors.py

Этот код реализует FastAPI-эндпоинт для создания записи о враче в базе данных с проверкой прав доступа.

Плюсы:

1. Правильная архитектура:
 - Разделение на схемы (schemas), модели (models) и зависимости.
 - Использование response_model для валидации вывода.
2. Проверка прав доступа:
 - Убеждается, что user_id действительно принадлежит врачу.

Минусы:

1. Нет обработки дубликатов → если врач с таким user_id уже есть, будет ошибка.
2. Синхронный код → если БД поддерживает async, лучше использовать async/await.

